Lecture 13

Generative Models and Universal Approximation Theorem

Elena Raponi | Neural Computing course

Universiteit
Leiden
The Netherlands

Index

Part 1

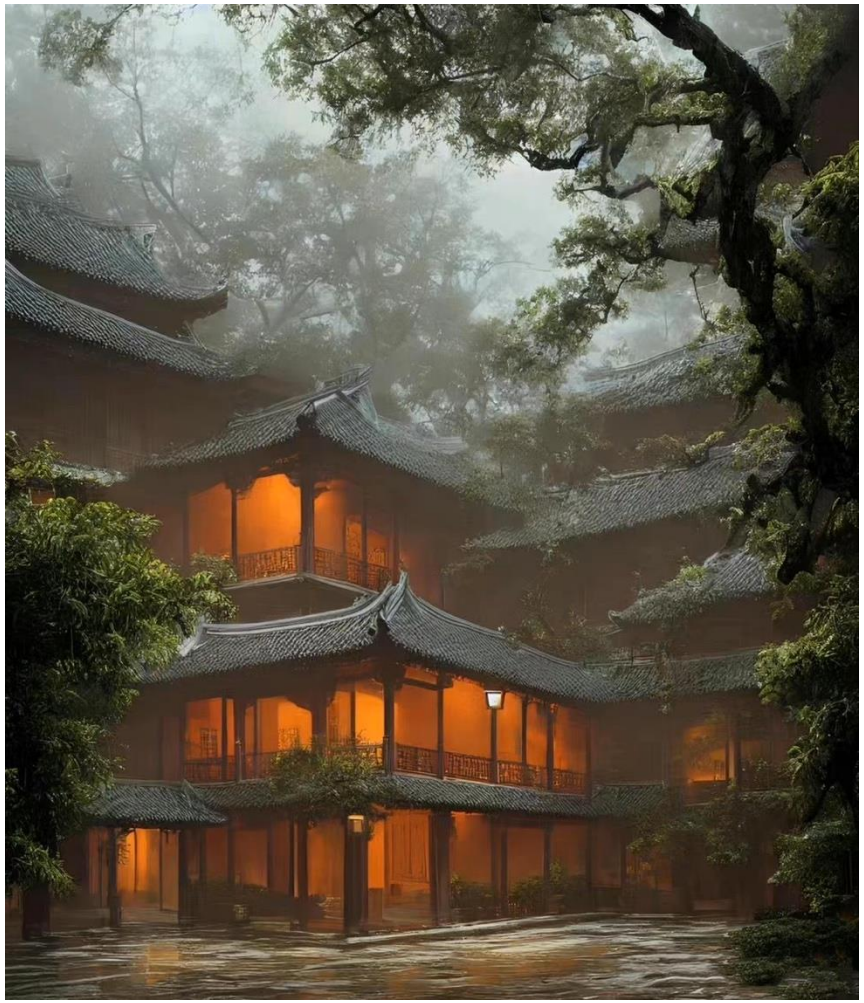
1. A Glimpse into Generative Models
2. Examples
3. Autoencoders
4. Generative Adversarial Networks

Part 2

1. Universal Approximation Theorem
2. Theorem Statement
3. Visual Proof

A Glimpse into Generative Models

Introduction



Which one is real?

Introduction

Introduction

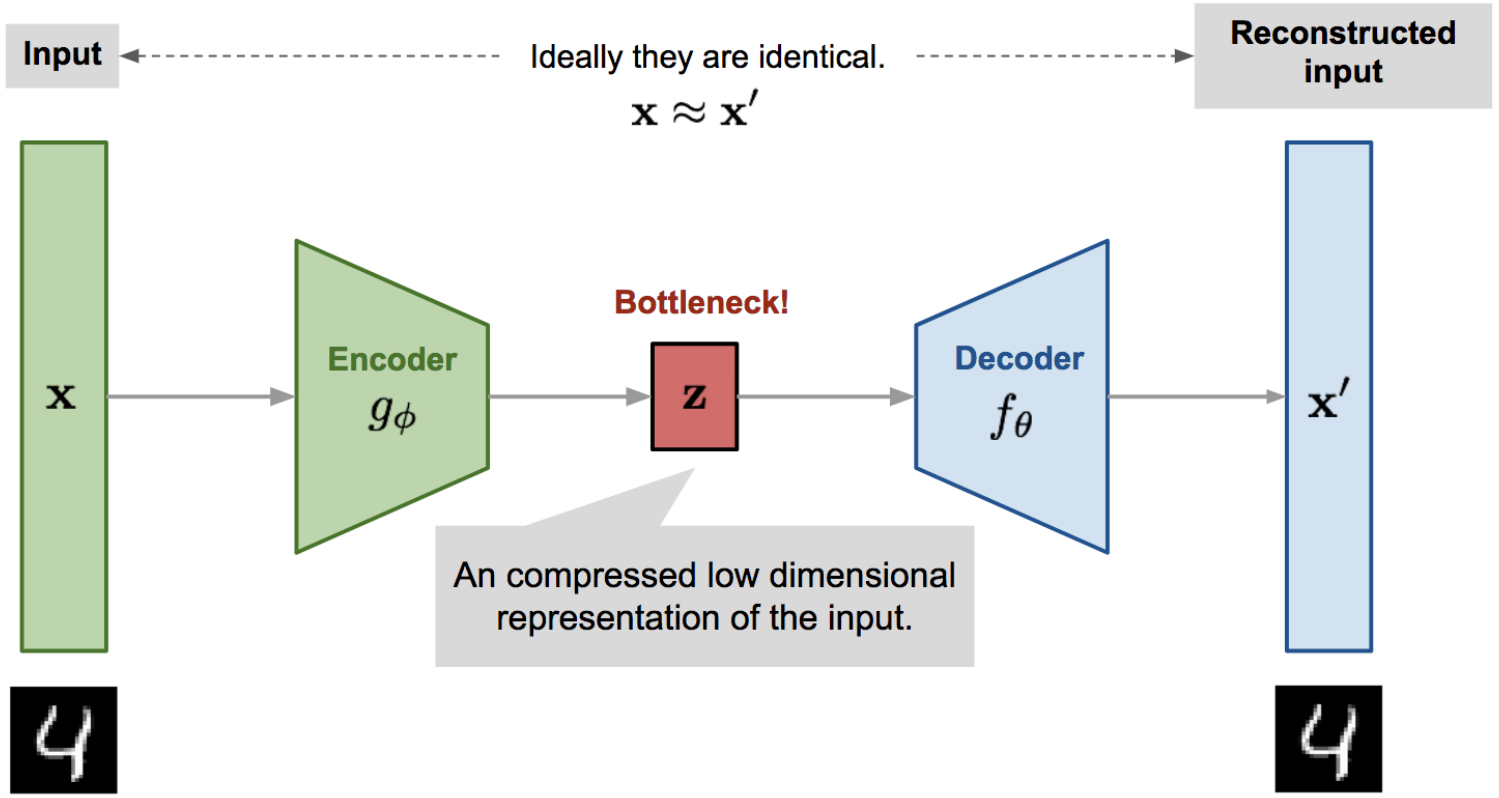
- Formally, generative modeling is unsupervised learning

- Create new forms of data learned from existing ones ***with novelty***
  - Image synthesis
  - Image captioning
  - Video generation
  - Language generation, e.g., GPT models
  - Generate reports, power point slides…
  - …

- But, how?

Autoencoder

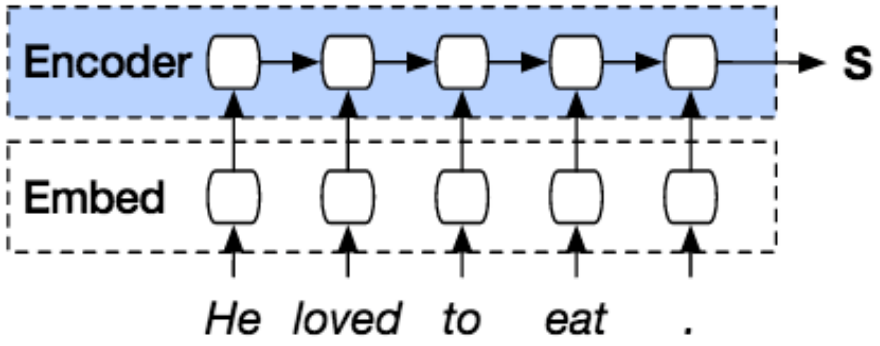
- Two (sub-)networks: encoder and decoder
  - **Encoder**: to obtain the *latent low-dimensional representation* of the data.
  - **Decoder**: reconstruct the data from the latent code.



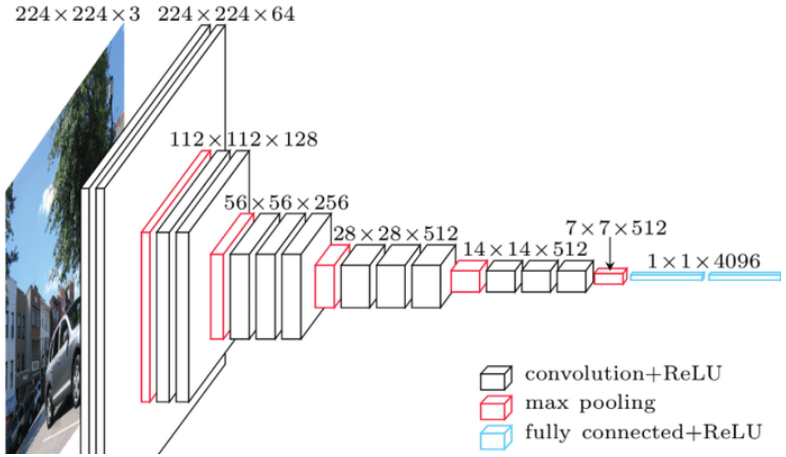
Autoencoder

- Encoder – typically a neural network
  - For image input, a CNN
  - For time-series input, a multi-layer LSTM
  - For tabular data, multi-layer perceptron
  - ϕ represents all trainable parameter therein, e.g., the weights and biases

Encoder of the Seq2Seq model



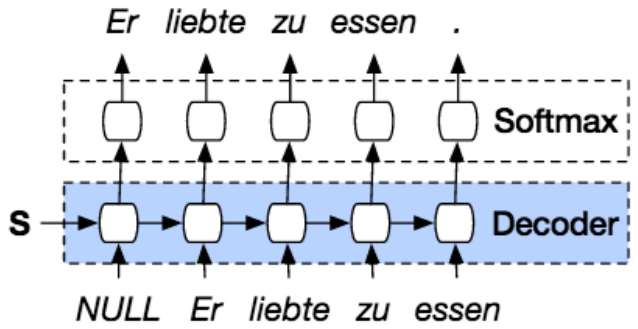
Encoder for image inputs

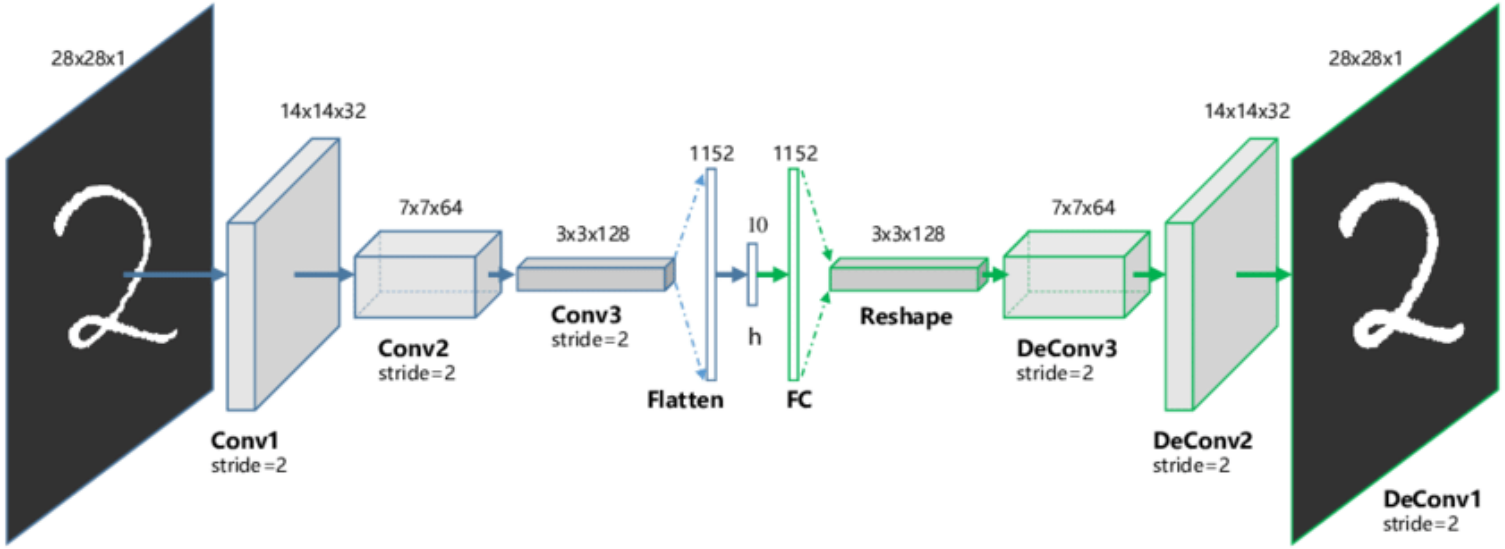


Autoencoder

- Decoder – another neural network
  - θ represents all trainable parameter therein, e.g., the weights and biases

Decoder of the Seq2Seq model:

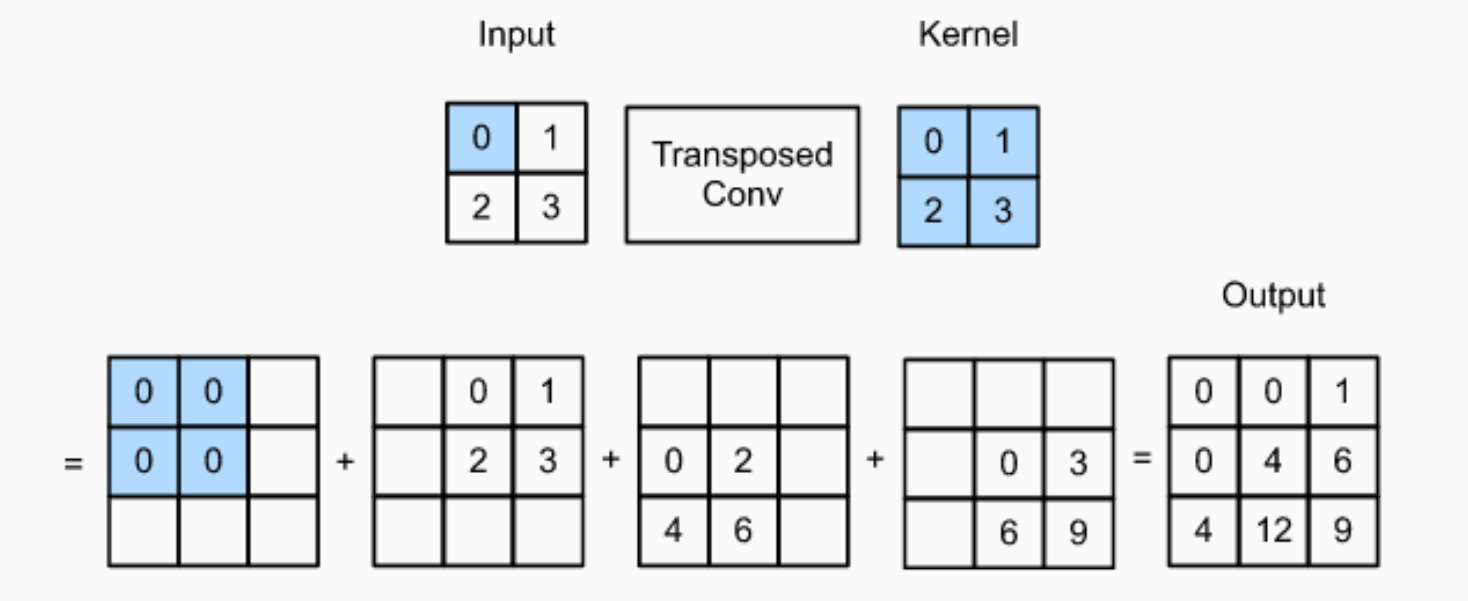




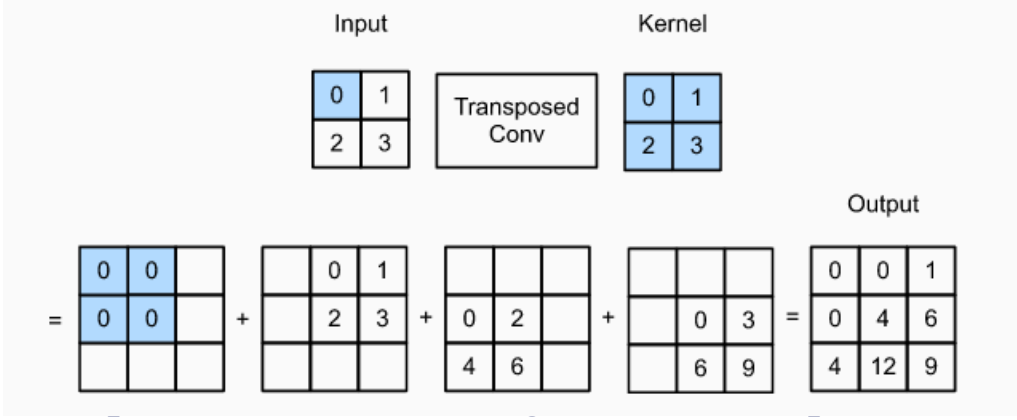
Autoencoder

Decoder for image data: how does “deconvolution” work?

- Two options:
  - Upsampling (interpolation)
  - *Transposed convolution*



Autoencoder



- **Standard Convolution:**
  - **Input:** An image (or feature map) and a filter (kernel).
  - **Process:** The filter slides over the image, performing element-wise a single value for each position.
  - **Output:** A smaller (or equal-sized) feature map, depending on padding and stride.

- **Transposed Convolution:**
  - **Input:** A feature map and a filter (kernel).
  - **Process:** Unlike standard convolution, transposed convolution spreads each input pixel value over a larger area by reversing the process. It can be visualized as inserting zeros between elements in the input and then applying the standard convolution.
  - **Output:** A larger feature map, effectively upsampling the input.

- **Purpose:**

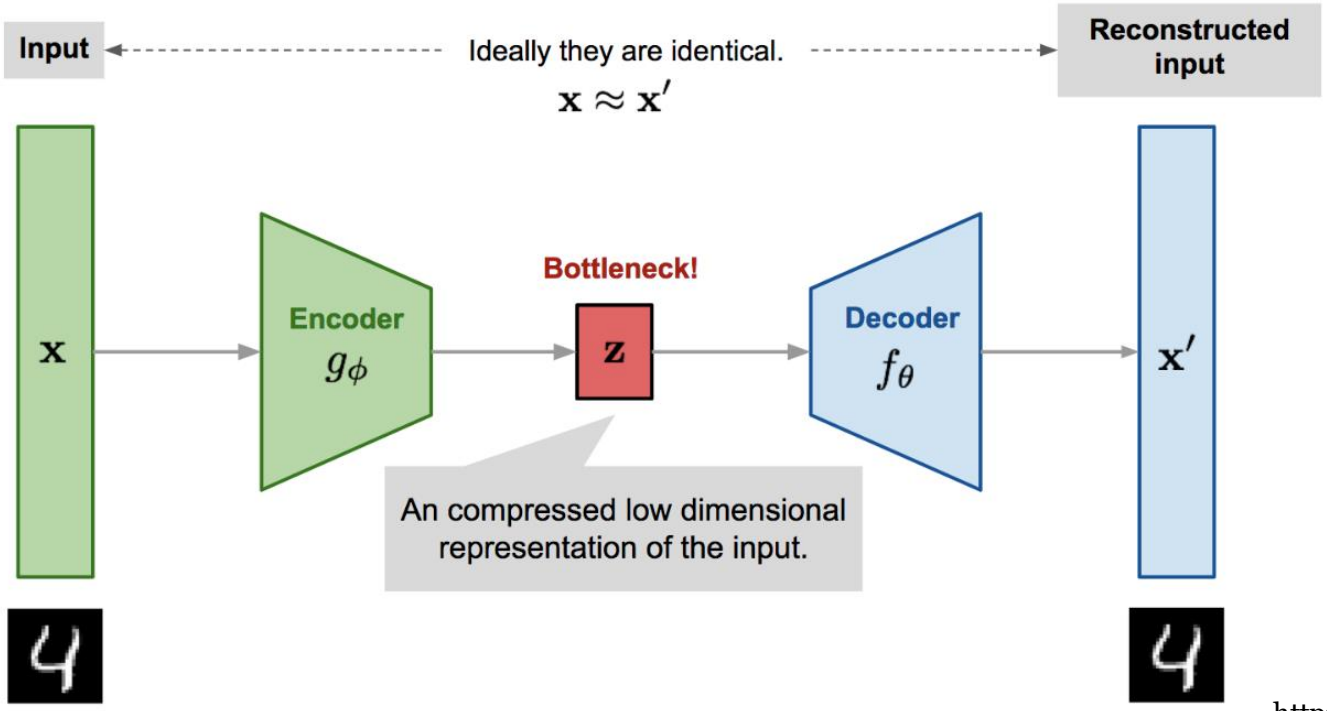
  To increase the spatial resolution of the input feature map, effectively "undoing" the downsampling done by standard convolutions.

Autoencoder

- Loss function - *reconstruction error*

$$\mathcal{L}_{\text{AE}}(\phi, \theta) = \frac{1}{n} \sum_{i=1}^{n} \left[\vec{x}^{(i)} - f_\theta \left(g_\phi \left(\vec{x}^{(i)} \right) \right) \right]^2$$

- Mathematically, we try to learn an identity function: $\text{id} \approx f_\theta \circ g_\phi$

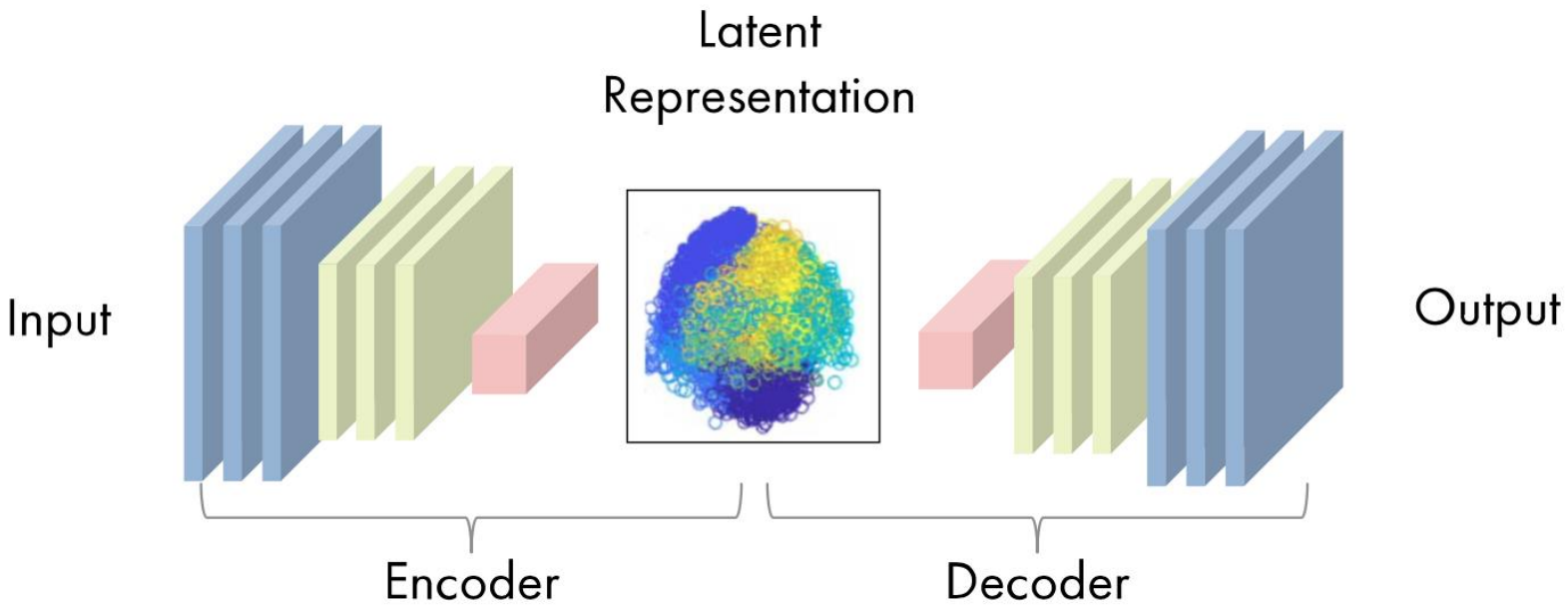


https://lilianweng.github.io/posts/2018-08-12-vae/

Autoencoder

Generate new data points:

- After training, sample a random point in the latent space
- Decode this random latent point
- A new image!



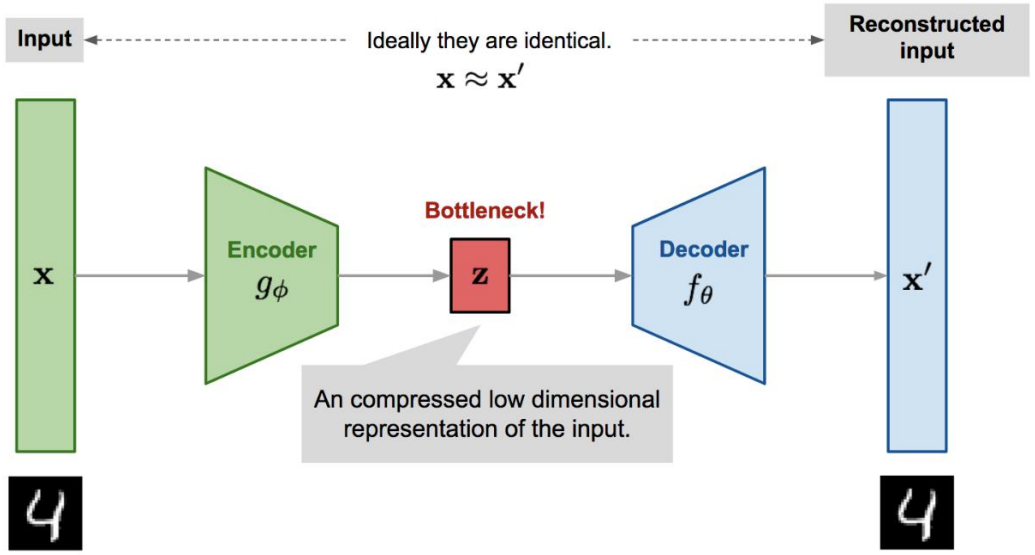
Autoencoder: Example

- Trained on many many images of human faces
- If we sample latent points continuously, we obtain nearly a “spectrum” of new faces with different styles.

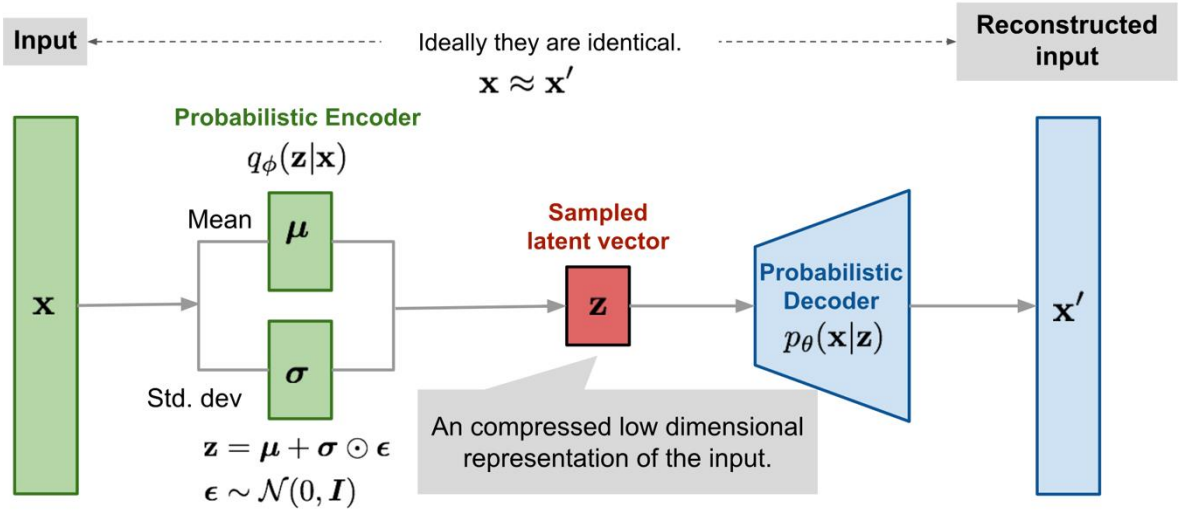
White, T. "Sampling generative networks: notes on a few effective techniques CoRR (2016)." *arXiv preprint arXiv:1609.04468*.

Variational Autoencoders (VAEs)

- Standard autoencoders learn a compressed latent representation z
- However, the latent space may be irregular or discontinuous
- Small changes in z may produce unrealistic outputs
- **Key Idea:** Instead of mapping an input to a single latent point, a VAE learns a **probability distribution.**



Autoencoder



Variational Autoencoders

https://lilianweng.github.io/posts/2018-08-12-vae/

Variational Autoencoders (VAEs)

Why is this useful?

- Smooth latent space
- Meaningful interpolation between samples
- Better generation of new unseen data

Loss function: Reconstruction error + Regularization of latent space (KL divergence)

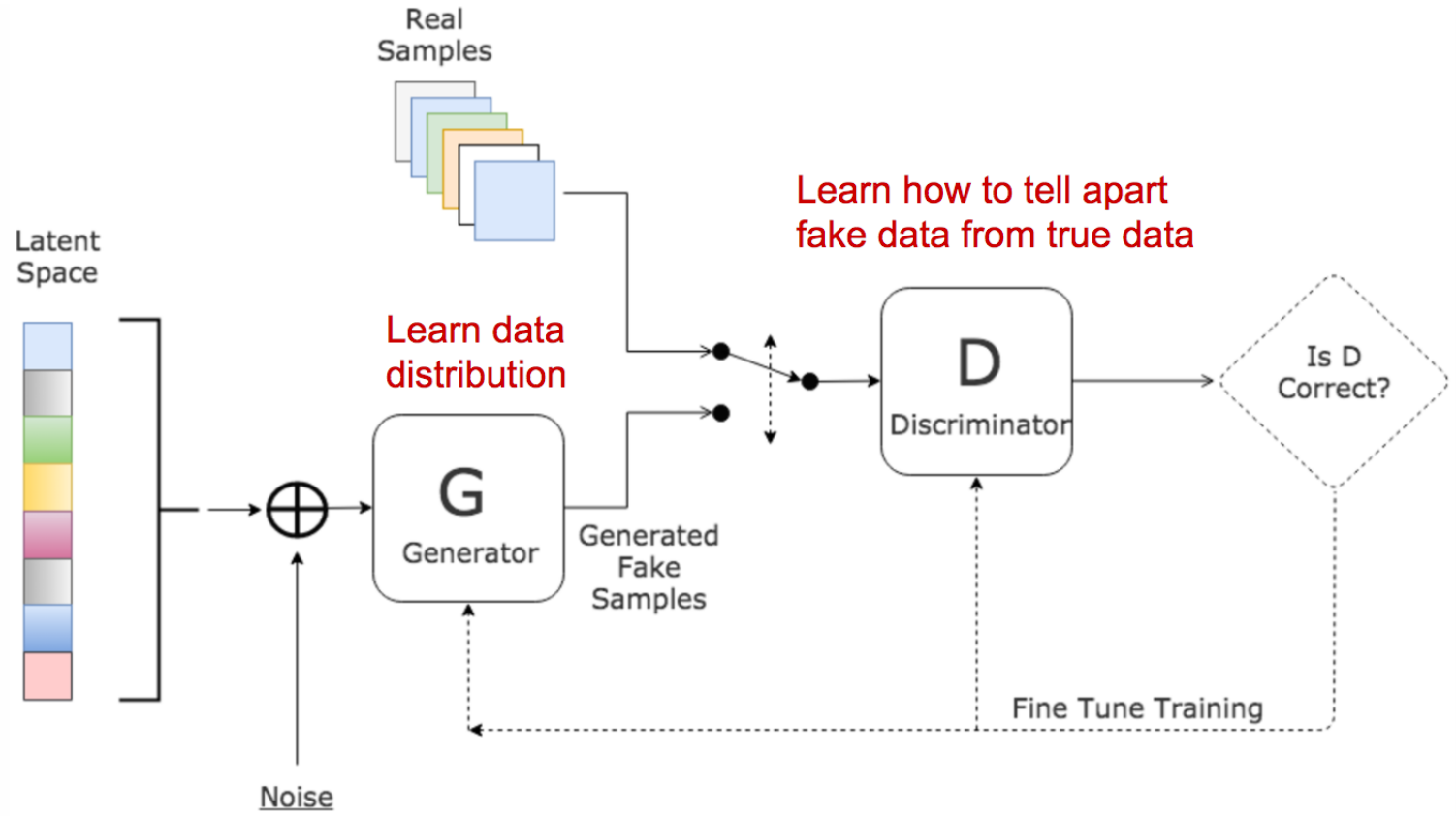
$$\mathcal{L}_{VAE} = \underbrace{\mathcal{L}_{reconstruction}}_{\text{Reconstruct input well}} + \underbrace{D_{KL}(q(z|x) \,||\, p(z))}_{\text{Keep latent space regular}}$$

The KL divergence measures:
"How different is the encoder's distribution from a standard normal distribution?"

https://lilianweng.github.io/posts/2018-08-12-vae/

Generative Adversarial Network (GAN)

Two (sub-)networks: a discriminator D and a generator G



https://www.kdnuggets.com/2017/01/generative-adversarial-networks-hot-topic-machine-learning.html

Generative Adversarial Network (GAN)

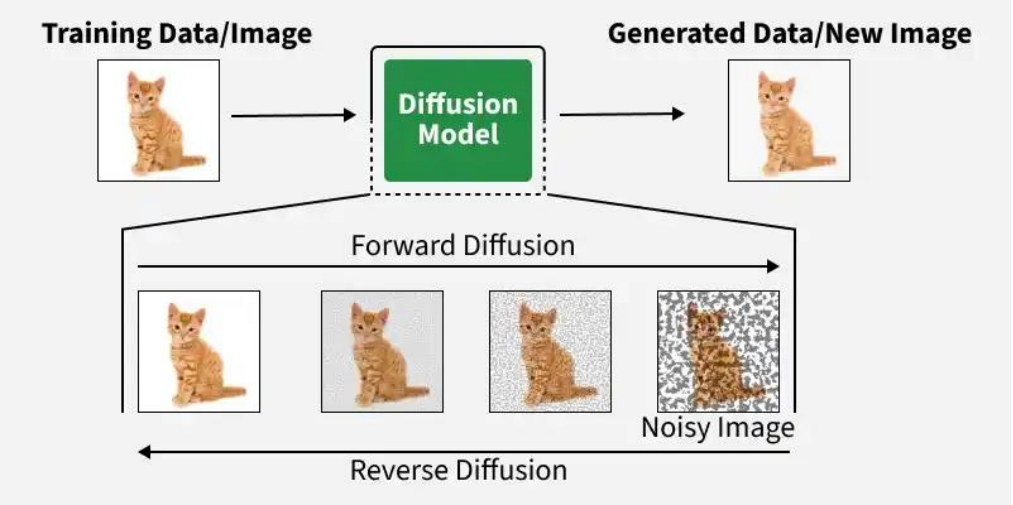
- A discriminator D (an image classifier) aims to determine if an image is real or synthetic.

- A generator G creates synthetic images from noise, trying to make them realistic enough to fool D.

- The process:
  1. G generates synthetic samples starting from random latent noise vectors.
  2. D receives a mix of real and synthetic images and attempts to distinguish between them.
  3. Based on D's success or failure, both D and G adjust their parameters to improve.
     - If D is correct, G improves to create better fakes.
     - If D is wrong, D learns to avoid similar mistakes.
  4. D is rewarded for correct predictions, while G is rewarded for D's errors.
- This iterative process continues until both networks reach an optimal balance.

Diffusion Models

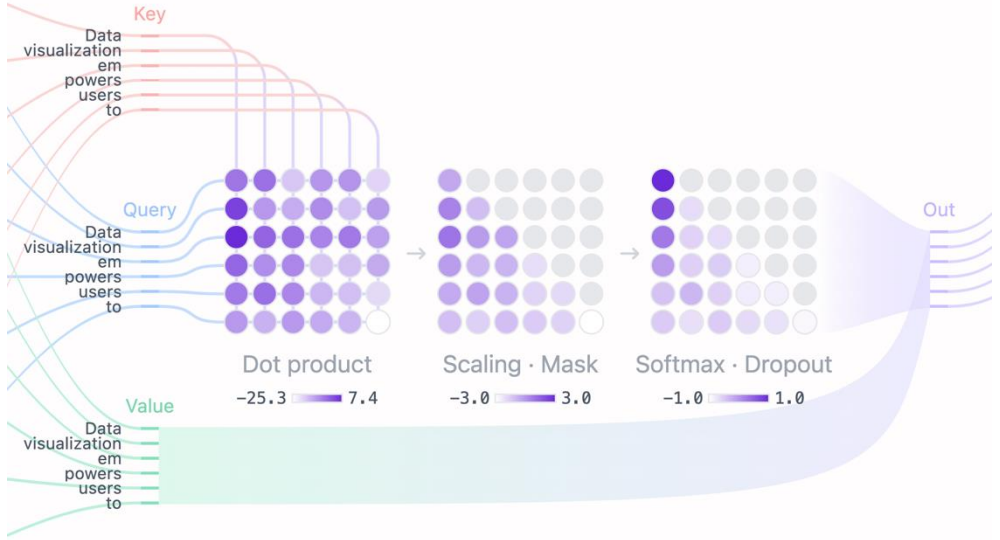
- Modern image generation is dominated by diffusion models much more than GANs.
- GANs are historically important, but diffusion models are today's state of the art.
- The model learns how to iteratively denoise random noise into meaningful data.



https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Transformers

- Transformers are the foundation of modern generative AI systems such as GPT, Gemini, Claude, Llama, etc.
- Instead of processing data sequentially like RNNs, transformers use an **attention mechanism** to determine which parts of the input are most relevant.



Chapter 12 of the Bishop & Bishop book "Deep Learning"

Universal Approximation Theorem

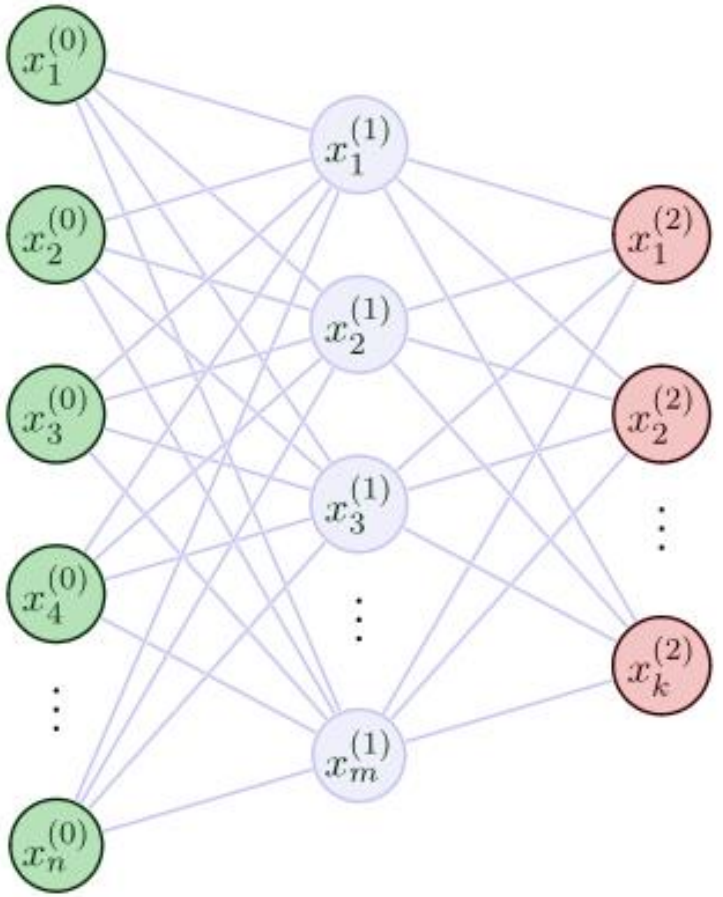
Theorem Statement

What does it actually mean that an ANN is a universal approximator?

The **Universal Approximation Theorem** states that:

Any continuous function $f: [0,1]^n \rightarrow [0,1]$
can be approximated with arbitrary precision by a neural network with at least 1 hidden layer with a finite number of weights.

To this end, the corresponding number of hidden nodes, number of learning iterations, weight parameters and biases need to be adapted individually for the to-be-approximated function.



Theorem Statement

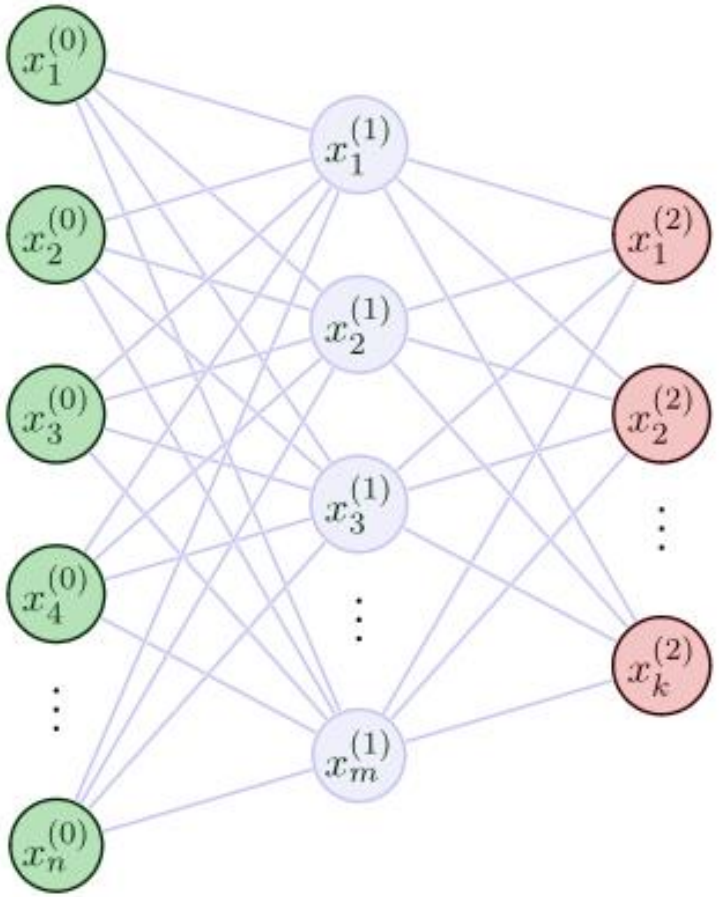
What does it actually mean that an ANN is a universal approximator?

The **Universal Approximation Theorem** states that:

Suppose we have a function $f(x)$ that we would like to compute within some desired accuracy $\epsilon > 0$.
It is guaranteed that by using enough hidden neurons we can always find a neural network whose output $g(x)$ satisfies

$$|g(x) - f(x)| < \epsilon$$

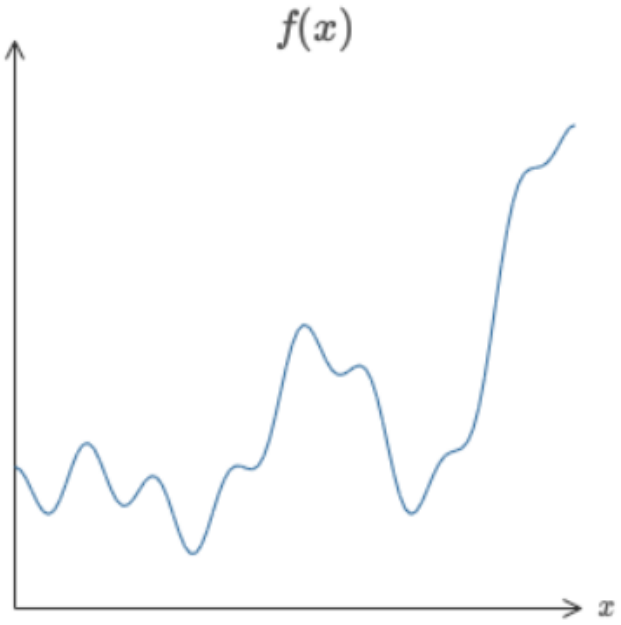
for all inputs x.



Visual Proof

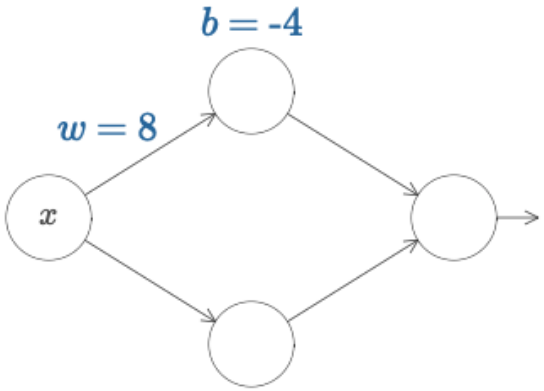
Let's consider:

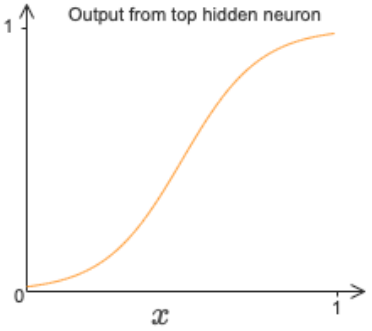
- A function with 1 input and 1 output



- A neural network with:
  - 2 hidden neurons
  - 1 output neuron
  - A sigmoid activation function

$$\sigma(wx + b), \text{ where } \sigma(z) \equiv \frac{1}{1 + e^{-z}}$$





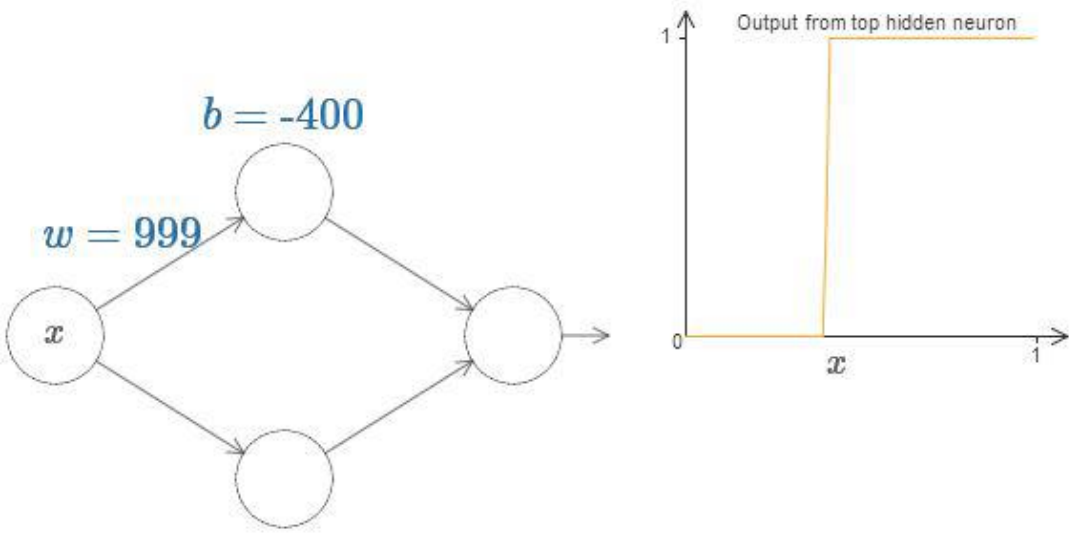
[Credits to: http://neuralnetworksanddeeplearning.com]

Visual Proof

Step 1

By operating on one of the neurons, generate a step function (easier to sum up)

We do this by fixing the weight w to be some very large value, and then setting the position of the step by modifying the bias.



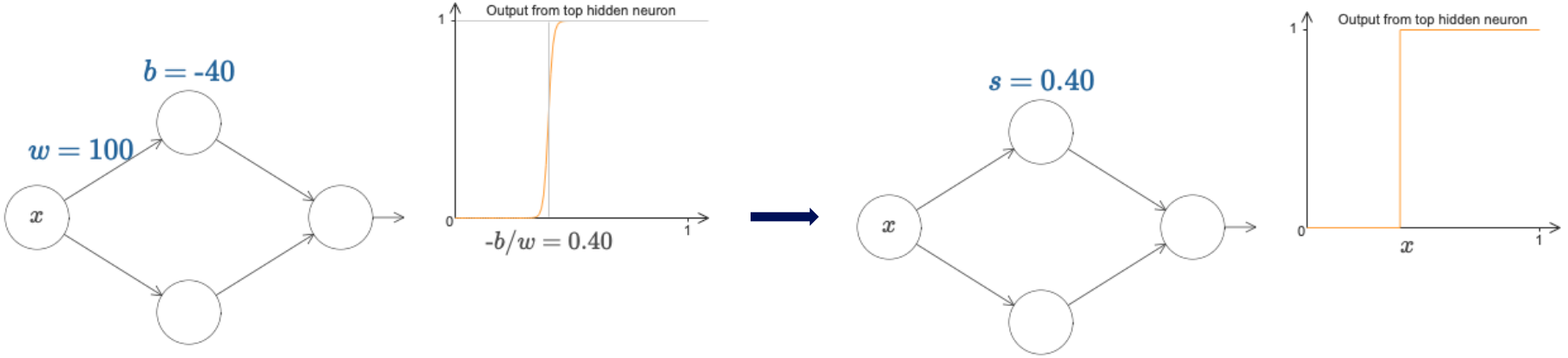
Note that the position of the step is proportional to b, and inversely proportional to w. In particular, the step is at position $s = -\frac{b}{w}$.

Visual Proof

Step 1

Note that the position of the step is proportional to b, and inversely proportional to w.
In particular, the step is at position $s = -b/w$.

From now on we will consider just a single parameter s.



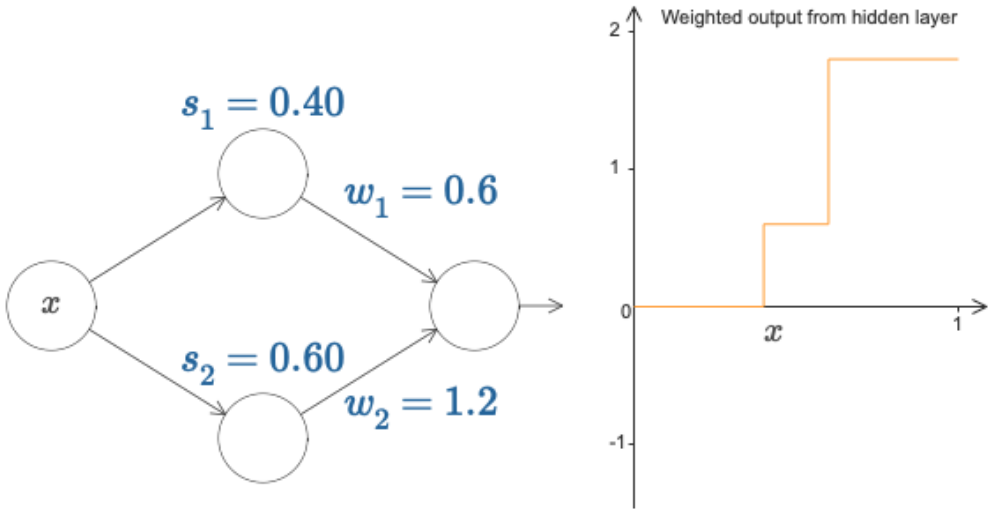
Visual Proof

Step 2

Let's focus on the whole network and suppose the hidden neurons are computing step functions parametrized by the step points s_1 and s_2.

Let's consider the weights w_1 and w_2 on the hidden-to-output connections.

We can compute the weighted output $w_1 a_1 + w_2 a_2$ from the hidden layer:

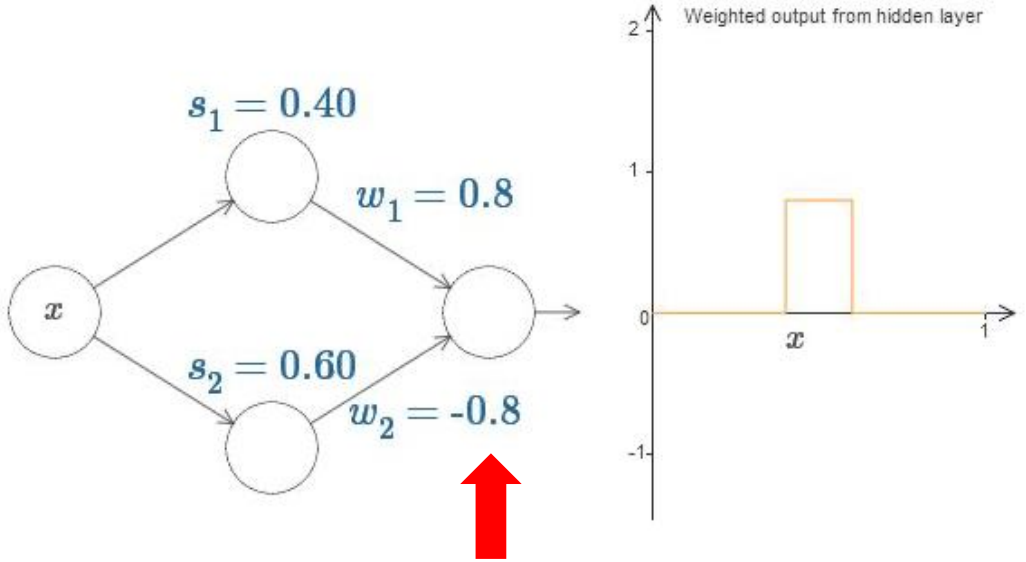


Visual Proof

Step 3

Working on the second weight, obtain a “bin” function, starting at point s_1 and ending at s_2.

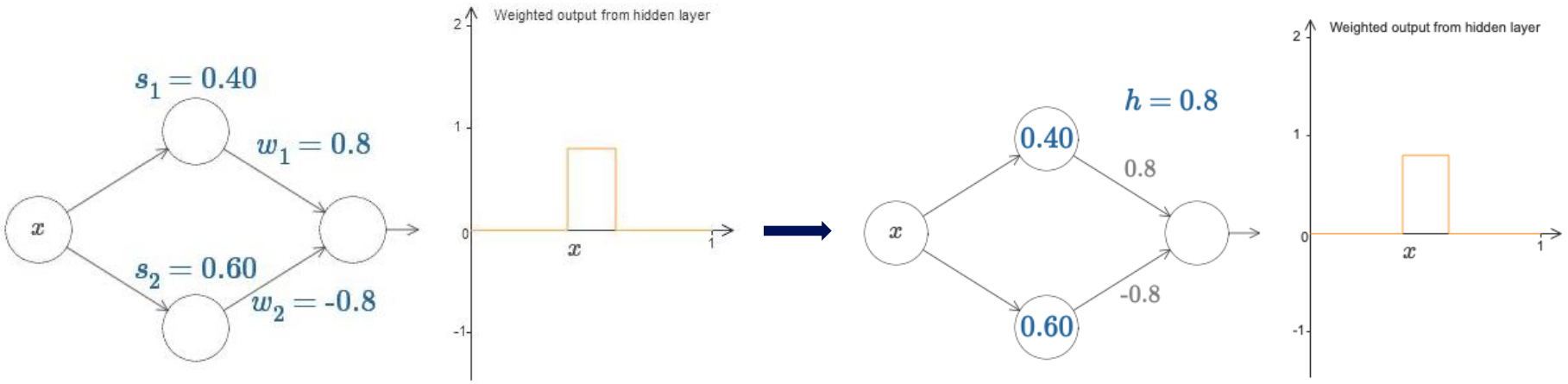
As illustrated below, by using the other neuron to make a step function, and setting opposing weights in the second layer, we can effectively approximate a bin and control its position, size and height.



Visual Proof

Step 3

We can hence use just one parameter $h = w_1 = -w_2$ to control both the weights!



Visual Proof

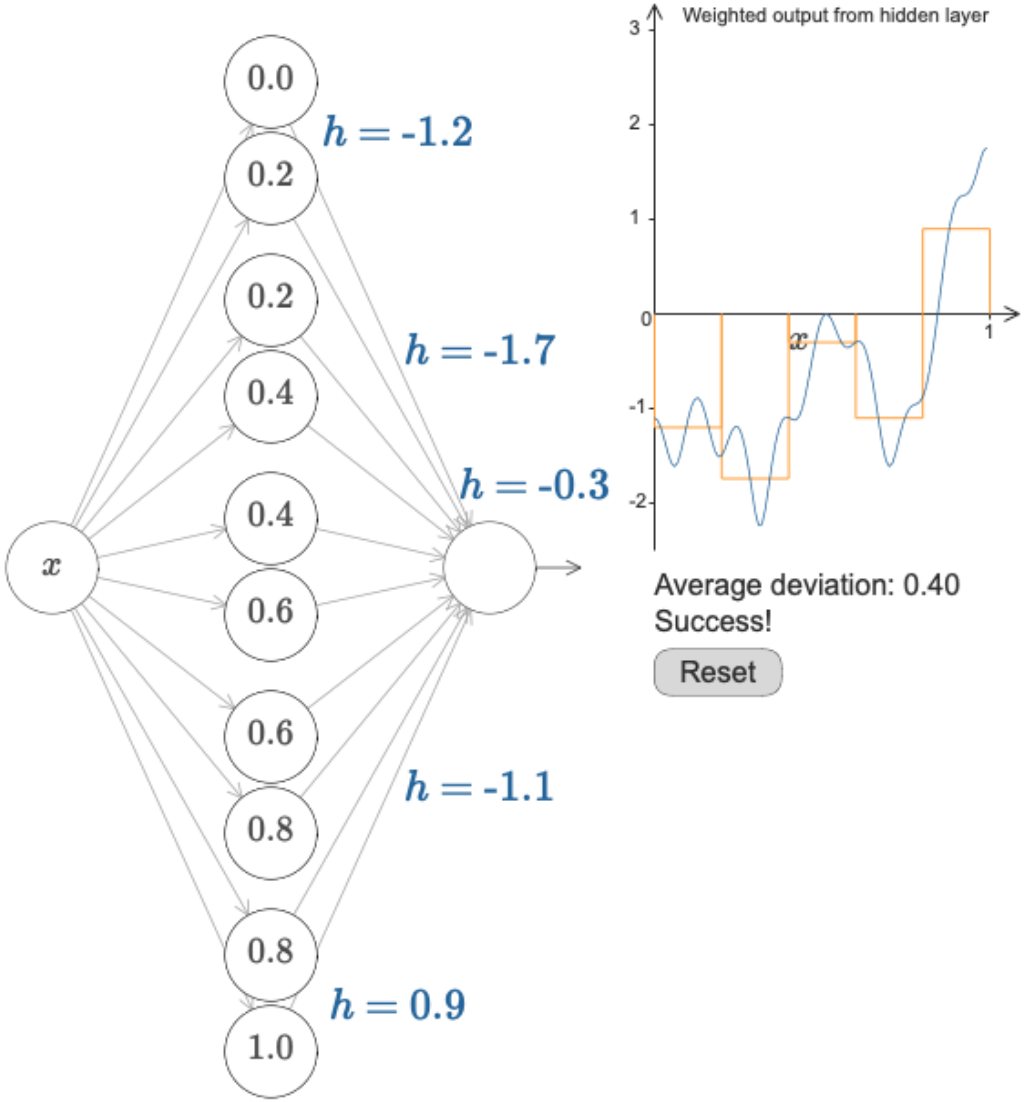
Step 4

By adding pairs of hidden neurons, we can create **multiple bins**.

More generally, if we divide the interval [0,1] into N subintervals and use N pairs of hidden neurons to set up peaks at different desired heights, we can generate a **histogram that discretizes the function**.

Playing with the heights and the number of bins, we can **approximate a function up to a desired precision**.

For example, if we set the average deviation between the goal function and the approximation the NN is computing to 0.4, we can get the approximation on the right.



Thank you – Any questions?

Next (LAST) class:
Online Q&A on
Wednesday, May 27,
15:15-17:00

e.raponi@liacs.leidenuniv.nl

Universiteit
Leiden
The Netherlands